

41039 Programming 1

Weeks 2 & 3: Choosing and Repeating



Dr. Xuan-Bach Le

le.xuan.bach@hcmut.edu.vn

Faculty of Computer Science and Engineering, HCMUT

September Session 2026

Outline

- 1 Types, Booleans and Expressions
- 2 Choosing in Java
- 3 switch and Fall-Through
- 4 Choosing in Python
- 5 Loops in Java
- 6 Nesting, Breaking, Continuing
- 7 Loops in Python
- 8 Arrays in Java
- 9 Lists in Python
- 10 Dictionaries, Sets and Tuples
- 11 Bundling Data Together

Where we were, and where we are going

Week 1 gave you one of the three tools.

- **State** — variables that hold values, and a program that runs top to bottom
- Which means: it does exactly the same thing every single time

These two weeks add the other two.

- **Branching** — the program can *choose* (Weeks 2–3, sections 1–4)
- **Repetition** — the program can *repeat* (sections 5–7)
- Then the structures worth repeating over: arrays, lists, dicts, sets, tuples

Everything here comes from the **Week 2 and 3 lessons on Ed** — eleven of them — in the UTS course format for 41039, as conducted at UTS by **Dr. Luke Mathieson**.

After today you have every tool a general-purpose program needs. Week 6 onwards adds no fourth idea.

Outline

- 1 Types, Booleans and Expressions
- 2 Choosing in Java
- 3 switch and Fall-Through
- 4 Choosing in Python
- 5 Loops in Java
- 6 Nesting, Breaking, Continuing
- 7 Loops in Python
- 8 Arrays in Java
- 9 Lists in Python
- 10 Dictionaries, Sets and Tuples
- 11 Bundling Data Together

Java is strongly, statically typed

- Variables have types. Data has types.
- Those types must be **compatible**, and **known at compile time**

Week 1 showed you one type error. There are others — and they all arrive *before* the program runs.

```
int a = Math.round("This isn't a number"); // no suitable method
int i = 3 / "Hi";                          // bad operand types
```

Two errors from one compile. Always start reading at the first one.

It is not only variables that have a type

- **Methods** have a return type — that `void` in `public static void main` means “returns nothing”
- **Expressions** have a type — the type of the value they evaluate to

The expression `10 + 3` has type `int`. So this is fine:

```
int i = 10 + 3;
```

And this is not:

```
boolean b = 10 + 3;  
// Type mismatch: cannot convert from int to boolean
```

Because expressions have types, you can use one anywhere a value of that type is expected.

boolean is its own type

Java has a boolean type for every true/false value, and it is **not compatible with the numeric types**.

Valid C++

```
bool b = true;  
int i = 10 + b;    // i is 11
```

Will not compile in Java

```
boolean b = true;  
int i = 10 + b;
```

A trade, not a flaw.

- C++ lets you write shorter code — and lets a typo compile
- Java refuses the shortcut — and catches the typo

Neither is “right”. But you should know which one you are typing into.

A language with no type rules

MY NEW LANGUAGE IS GREAT, BUT IT HAS A FEW QUIRKS REGARDING TYPE:

```
[1]> 2+2
=> 4
[2]> "2" + []
=> [2]
[3]> (2/0)
=> NaN
[4]> (2/0)+2
=> NaN
[5]> "" + ""
=> ""
[6]> [1,2,3]+2
=> FALSE
[7]> [1,2,3]+4
=> TRUE
[8]> 2/(2-(3/2+1/2))
=> NaN.00000000000000013
[9]> RANGE(" ")
=> (" ", " ", " ", " ", " ", " ")
[10]> +2
=> 12
[11]> 2+2
=> DONE
[12]> RANGE(1,5)
=> (1,4,3,4,5)
[13]> FLOOR(10.5)
=> 1
=> 1
=> 1
=> 10.5
```

xkcd 1537, *Types* — Randall Munroe, CC BY-NC 2.5

- $2 + "2"$ gives "4"
- $[1,2,3] + 2$ gives FALSE
- $[1,2,3] + 4$ gives TRUE

Expressions whose *type* you cannot predict — so you cannot predict their value either.

This is the world Java's “cannot convert from int to boolean” is protecting you from.

The operators that produce a boolean

A lone `boolean` is dull. They become useful as the type of a whole *expression*.

Operator	Means	Takes
< <= > >=	less, less-or-equal, greater, greater-or-equal	two numbers
== !=	equal to, not equal to — note the <i>two</i> equals signs	two of any type
&&	and — true only when both sides are true	two booleans
	or — true when either side is true	two booleans
!	not — flips true and false	one boolean

The classic typo: `=` assigns, `==` compares. In Java `if (x = 5)` is usually *caught* — the assignment's type is `int`, and an `if` needs a `boolean`. In C it compiles, and has ruined many afternoons.

The type system saving you again.

Combining comparisons

```
Scanner sc = new Scanner(System.in);
System.out.print("Enter a number: ");
int i = sc.nextInt();

boolean b = i > 10 && i < 20;

System.out.println("Bigger than 10 but less than 20? " + b);
```

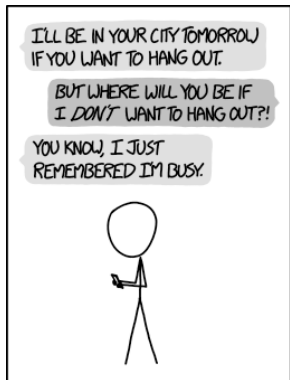
- `i > 10` is an expression of type `boolean`
- So is `i < 20`
- So `&&` can take both, and the result is `boolean` too — which is exactly why it can be stored in `b`

This is really exciting. It becomes useful when we choose what to do with it.

Outline

- 1 Types, Booleans and Expressions
- 2 Choosing in Java
- 3 switch and Fall-Through
- 4 Choosing in Python
- 5 Loops in Java
- 6 Nesting, Breaking, Continuing
- 7 Loops in Python
- 8 Arrays in Java
- 9 Lists in Python
- 10 Dictionaries, Sets and Tuples
- 11 Bundling Data Together

Why we need else



WHY I TRY NOT TO BE
PEDANTIC ABOUT CONDITIONALS.

xkcd 1652, *Conditionals* — Randall Munroe, CC BY-NC 2.5

The pedant is asking for the **else branch**.

- Everyday English “if X, then Y” quietly implies “otherwise, *not* Y”
- if (cond) { ...} says **nothing at all** about the false case

That gap is this entire section.

Two minutes from now you will write a bug that lives in exactly that gap.

The if statement

Now that a program can say whether something is true, it can use that to decide what to do next.

```
if (<condition>) {  
    <what to do when the condition is true>  
}
```

```
if (i < 100) {  
    System.out.println("The number is less than 100.");  
}
```

The <condition> must be a boolean expression — which is what the whole of section 1 was for.

State, plus a choice. That is two of the three tools.

Two ifs are not an if-else

What about when the condition is false? The obvious move is a second if.

```
if (i < 100) {  
    System.out.println("The number is less than 100.");  
    i = 101;  
}  
  
if (i > 100) {  
    System.out.println("The number is greater than 100.");  
}
```

Enter 50. What prints?

Both lines. Ask the room before revealing the next slide.

Two problems, and the second one is nastier

- ❶ **What about exactly 100?** Neither condition is true and the program says nothing at all. Separate conditions need separate care — a good place to introduce a logic bug quietly.
- ❷ **The first block changed *i*.** By the time the second `if` is tested, `i` is 101 — so *both* messages print, for a single input.

An `else` block fixes both, because the branches become genuinely exclusive: **exactly one of them runs**.

```
if (<condition>) {  
    <code when the condition is true>  
}  
else {  
    <code when the condition is false>  
}
```

“Exactly one” is a guarantee. Two `ifs` give you no guarantee at all.

Many conditions: else if

Nesting an if-else inside an else works, and gets ugly fast. When you are choosing on one value, lay the cases out at the same level:

```
if (i < 100) {  
    System.out.println("The number is less than 100.");  
}  
else if (i == 100) {  
    System.out.println("The number is 100.");  
}  
else {  
    System.out.println("The number is greater than 100.");  
}
```

Tested **one after another** until one is true, or the final else is reached.

“Take care to branch the right way on equality.”

Brian W. Kernighan & P. J. Plauger, *The Elements of Programming Style*, 2nd ed., 1978

Which means order matters — and you can use that deliberately.

Order decides the winner

```
if (i == 42) {  
    System.out.println("You've found the answer");  
}  
else if (i < 100) {  
    System.out.println("The number is less than 100.");  
}  
else {  
    System.out.println("The number is 100 or greater.");  
}
```

- Enter 42: only the **first** block runs — the rest are never even looked at
- A specific case placed *before* an overlapping general case wins
- Move `i == 42` to the bottom and it is unreachable — and nothing warns you

The compiler checks your types. It does not check your logic.

Outline

- 1 Types, Booleans and Expressions
- 2 Choosing in Java
- 3 switch and Fall-Through
- 4 Choosing in Python
- 5 Loops in Java
- 6 Nesting, Breaking, Continuing
- 7 Loops in Python
- 8 Arrays in Java
- 9 Lists in Python
- 10 Dictionaries, Sets and Tuples
- 11 Bundling Data Together

When every test is “is it equal to...”

Say x being 1, 5, 87 or 120 means one thing, and anything else means another:

```
if (x == 1 || x == 5 || x == 87 || x == 120) { /* one thing */ }  
else { /* another */ }
```

Bearable for four cases. Painful for twenty — and worse if each case does something *different*. **A switch is a compacted else if ladder, restricted to equality.**

- One readable line per case, easy and safe to add to
- You can switch on a whole *expression*, not just a variable
- Switchable types: byte, short, char, int, String, plus enums and object types via pattern matching

If the conditions cannot be reduced to equality checks, switch is simply not available.

break, and what happens without it

```
int x = 2;

switch (x) {
    case 1 : System.out.println("The value is 1.");
    case 2 : System.out.println("The value is 2.");
    case 3 : System.out.println("The value is 3.");
    default : System.out.println("The value is not 1, 2 or 3.");
}
```

Predict the output before you read on.

Three lines — not one.

Fall-through

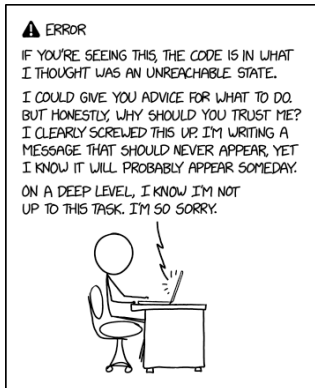
A switch finds the matching case, **starts running there**, and keeps going — through every case below it — until it hits a break or the end of the block.

```
The value is 2.  
The value is 3.  
The value is not 1, 2 or 3.
```

- It starts *at the match*, not at the top — so “The value is 1.” never prints
- And then nothing stops it

A missing break is one of the most common bugs in this entire subject.

The case you were certain could not happen



NEVER WRITE ERROR MESSAGES TIRED.

xkcd 2200, *Unreachable State* — Randall Munroe, CC BY-NC 2.5

This is the argument for always writing a default case — and, back in section 2, a final else.

- The branch you are *sure* is impossible is the one in the bug report
- “That can never happen” is a prediction, not a guarantee

Handle the impossible case. It costs you two lines.

Fall-through on purpose

Used deliberately, it is how you group cases that share an action:

```
switch (x) {  
    case 1 :  
    case 5 :  
    case 87 :  
    case 120 :  
        System.out.println("One of the special values.");  
        break;  
    default :  
        System.out.println("Just an ordinary number.");  
}
```

switch or if-else if? Context decides. A column of one-line cases is easier to *extend correctly* than a three-line boolean condition — but only equality checks are available.

Remember this shape. It comes back in section 8 in a genuinely strange way.

Outline

- 1 Types, Booleans and Expressions
- 2 Choosing in Java
- 3 switch and Fall-Through
- 4 Choosing in Python
- 5 Loops in Java
- 6 Nesting, Breaking, Continuing
- 7 Loops in Python
- 8 Arrays in Java
- 9 Lists in Python
- 10 Dictionaries, Sets and Tuples
- 11 Bundling Data Together

Same idea, less punctuation

Conditionals in Python work essentially as they do in Java, with mildly different syntax.

```
print("Enter a number:")
x = int(input())

if x < 0:
    print("You entered a negative number.")
elif x > 0:
    print("You entered a positive number.")
else:
    print("You entered zero.")
```

- elif, not else if
- A colon, not braces — and **indentation** does the work { } does in Java
- if (1 < 2): works, but if 1 < 2: is the preferred Python style

Once you have the idea in one language, every imperative language works the same. Only the syntax changes.

match — Python's switch, sort of

Before 3.10, Python had nothing like `switch` — only a hack with dicts that returns known values rather than running code. Since **version 3.10** there is structural pattern matching:

```
match <subject>:
    case <pattern 1>:
        <action 1>
    case <pattern 2>:
        <action 2>
    case _:
        <default action>
```

Notice what is missing: there is no `break`.

Because there is no fall-through. Only the matching block runs.

No fall-through changes the idiom

Java's stacked-label trick from section 3 is gone. In its place, | gives one case several values:

```
match x:
    case 1 | 5 | 87 | 120:
        print("One of the special values.")
    case _:
        print("Just an ordinary number.")
```

	Java switch	Python match
Fall-through	yes — needs break	no — never
Grouping	stacked case labels	case a b c:
Default	default :	case _:

Which starts to look like an if-elif again. The line between them is genuinely fuzzy.

Outline

- 1 Types, Booleans and Expressions
- 2 Choosing in Java
- 3 switch and Fall-Through
- 4 Choosing in Python
- 5 Loops in Java
- 6 Nesting, Breaking, Continuing
- 7 Loops in Python
- 8 Arrays in Java
- 9 Lists in Python
- 10 Dictionaries, Sets and Tuples
- 11 Bundling Data Together

Repetition is booooooring

A program without a loop and a structured variable isn't worth writing.

Alan Perlis, *Epigrams on Programming*, 1982

To print 1 to 10 with what you know so far, you write ten `println` lines.

```
System.out.println(1);  
System.out.println(2);  
System.out.println(3);    // ... and so on, seven more times
```

To print 1 to 1000 you write a thousand. Who is going to check that code?

And that is still the easy version.

The version that is not merely tedious

What if the number changes every time the program runs?

- The user picks it
- You cannot write out lines for a number you do not know

Until you have a loop, that program is not tedious to write. It is **impossible**.

“A cycle of operations, then, must be understood to signify any set of operations which is repeated more than once.”

Ada Lovelace, Note E to her translation of Menabrea's *Sketch of the Analytical Engine*, 1843

A loop performs the same basic action until some stopping condition is reached.

The while loop

```
while (<condition>) {  
    <code>  
}
```

Repeats for as long as the condition is true — which, like an if, must be a boolean expression.

```
System.out.print("How many numbers would you like to print? ");  
int n = sc.nextInt();  
  
int i = 1;  
while (i <= n) {  
    System.out.println(i);  
    i++;  
}
```

Three moving parts: **set a variable up, test it, change it.**

Only the condition changed from the fixed version: `i <= 10` became `i <= n`.

Forget the third part and

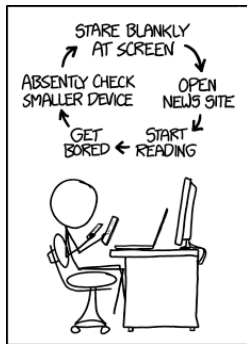
...the condition never becomes false.

The loop never ends.

- On Ed, and on the subject website, there is **no way to stop a running program** — you reload the page
- This is not a rare beginner mistake. It is *the* loop mistake, and you will make it more than once
- Delete the `i++` once, deliberately, so you recognise what it looks like

Every loop you write: ask “what makes this stop?” before you run it.

What makes this stop?



Read that as a loop and go looking for the exit condition.

There isn't one.

- A while whose condition never turns false
- A for whose variable never reaches its bound

xkcd 1411, *Loop* — Randall Munroe,
CC BY-NC 2.5

Ask it of every loop, every time, before you press Run.

The for loop

A for loop gathers those same three parts into one line, in order:

```
for (<initialisation>; <condition>; <modification>) {  
    <code>  
}
```

```
for (int i = 1; i <= n; i++) {  
    System.out.println(i);  
}
```

Read it as “*for i from 1 to n in steps of 1*”. Change the details and you change the pattern:

- `for (int i = 2; i <= n; i += 2)` — the even numbers
- `for (int i = 1; i <= n; i += 2)` — the odd ones

A little more compact than the `while`, and the three statements are all still there.

while vs for: which is best?

Neither. They are functionally equivalent.

A while as a for

```
int i = 1;
for (; i <= 10; ) {
    System.out.println(i);
    i++;
}
```

Java allows empty statements. An infinite `while (true)` becomes `for (;;)` .

A for as a while

```
int count = 0;
int i = 2048;
while (i > 1) {
    count++;
    i /= 2;
}
```

Just unpack the three parts and put them in the right places.

Identical to the machine. So the choice is about what you tell the reader.

So which one, when?

Use	When	Because
for	the number of iterations is driven by an index	the syntax forces you to state all three parts, and a reader takes in the whole pattern at a glance
while	you do not know how many steps it will take	user input, or a condition that is not naturally an index

The one case that really is while:

```
while (true) {  
    System.out.print("Enter 1 to stop: ");  
    if (sc.nextInt() == 1) break;  
}
```

Loops are for the computer. Choosing between them is for the human reading it next.

For-each: walking a collection

```
for (<type> <name> : <iterable collection>) {  
    <code>  
}
```

Read it as “*for each element in the collection*”. It hands you the elements one at a time — you never touch an index, so you cannot get the index wrong.

```
int[] a = {1, 3, 5, 7};  
  
for (int i : a) {  
    System.out.println(i);  
}
```

What counts as an “iterable collection”?

- **Arrays**, and anything that implements `Iterable`
- Which is most things in the Java library with a linear order

Notably *not* a `String`.

The one that does not compile

```
String s = "This won't work";  
  
for (char c : s) {  
    System.out.println(c);  
}
```

Can only iterate over an array or an instance of `java.lang.Iterable`

- The error message states the rule exactly
- Changing `char` to `String` does not help — the problem is **what you are walking**, not what you pull out of it

Read the error message. It is more helpful than it looks.

A for-each is syntactic sugar

Arrays and Iterables hand over their elements in a standard way, so the compiler writes the ordinary loop for you. The for-each unpacks to:

```
int[] a = {1, 3, 5, 7};  
  
for (int index = 0; index < a.length; index++) {  
    int i = a[index];  
    System.out.println(i);  
}
```

“Civilization advances by extending the number of important operations which we can perform without thinking about them.”

Alfred North Whitehead, *An Introduction to Mathematics*, 1911

Nothing new happens at run time. It is the same loop, with the index hidden.

do-while: test afterwards

Both loops so far check the condition *before* the first pass. Sometimes you need to run the body first — most often because you need the input before you can judge it.

```
do {  
    <code>  
}  
while (<condition>);    // note the semicolon
```

With a plain `while`, the prompt-and-read has to appear **twice** — once before the loop and once inside it.

“Let the machine do the dirty work.”

Brian W. Kernighan & P. J. Plauger, *The Elements of Programming Style*, 2nd ed., 1978

Once you are writing exactly the same code in two places, look for a better way.

Input validation, the tidy way

```
Scanner sc = new Scanner(System.in);

int n;
do {
    System.out.print("Enter a number: ");
    n = sc.nextInt();
    if (n != 6) System.out.println("That's not 6!");
}
while (n != 6);
```

- The body always runs **at least once** — which is the whole point
- The relevant code appears once, so a change happens in one spot
- Note `int n;` is declared *outside*, or the condition could not see it

When we get to methods, this gets better again.

Outline

- 1 Types, Booleans and Expressions
- 2 Choosing in Java
- 3 switch and Fall-Through
- 4 Choosing in Python
- 5 Loops in Java
- 6 Nesting, Breaking, Continuing
- 7 Loops in Python
- 8 Arrays in Java
- 9 Lists in Python
- 10 Dictionaries, Sets and Tuples
- 11 Bundling Data Together

Loops inside loops are not a new idea

“...there is a recurring group of one or more cycles; that is, a cycle of a cycle, or a cycle of cycles.”

Ada Lovelace, Note E, 1843

- She wrote that about a machine that was never built, a century before the first computer ran
- The same Note E defines a plain loop as “any set of operations which is repeated more than once”
- Nesting was in the idea from the very beginning

You are about to write a cycle of cycles.

Loops inside loops

Once you are inside a block you can write any statement — including another loop.

```
for (int i = 2; i < n; ++i) {  
    boolean isPrime = true;  
  
    for (int j = 2; j < i; ++j) {        // nested loop here  
        if (i % j == 0) isPrime = false;  
    }  
  
    if (isPrime) System.out.println(i + " is prime.");  
}
```

Note the inner loop's condition: `j < i` uses the **outer** loop's variable, so the inner loop does a different amount of work each time around.

Nesting is how you handle “for every x, for every y...”.

Two dimensions

The same shape walks a two-dimensional array — and the rows need not be the same length:

```
int[][] data = {{1,2,4},{7,8},{7,3,7},{9,5,2,2,5}};

for (int i = 0; i < data.length; ++i) {
    for (int j = 0; j < data[i].length; ++j) {
        System.out.print(data[i][j] + " ");
    }
    System.out.println();
}
```

- The inner bound is `data[i].length`, not one fixed number
- Three dimensions is one more level of nesting; matrix multiplication is the classic triple nest

A variable declared in a for header is local to that loop — so `i` and `j` can be reused later.

Loops with almost no body

Only the *middle* part of a for header is restricted — it must evaluate to a boolean. The other two can be almost any expression.

```
int sum = 0;

for (int i = 0; i <= 10; sum += i++);

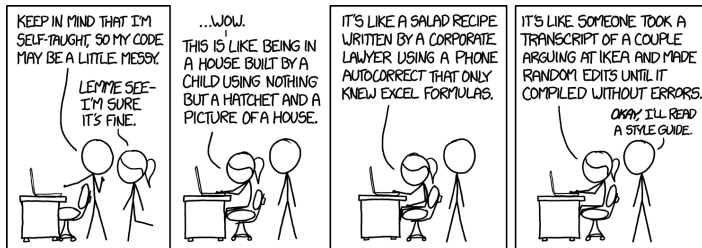
System.out.println(sum);    // 55
```

The whole loop lives in its own header, and the body is a bare semicolon.

Do not take this as an invitation.

Short is not the same as clear

- Cramming everything into the smallest space is not a programming strategy
- Outside the [Obfuscated C Code Contest](#), where it is the entire point
- Code is read far more often than it is written — including by you, next month



xkcd 1513, *Code Quality* — Randall Munroe, CC BY-NC 2.5

The genuinely useful version of this is much quieter.

A loop body can be a single statement

No braces needed. Combined with a for-each, that reads beautifully:

```
int[] values = {3, 6, 1, 8, 3, 8, 9, 0, 3};  
int sum = 0;  
  
for (int n : values) sum += n;  
  
System.out.println(sum);
```

- Compact *and* readable — which is the combination worth chasing
- “Short” is not the goal. “Clear” is the goal, and short is sometimes how you get there

“Write clearly — don’t be too clever.”

Brian W. Kernighan & P. J. Plauger, *The Elements of Programming Style*, 1978 — the book’s first rule

Compare with the previous slide. Same language, opposite instinct.

Finishing early: break

You met `break` in a `switch`. In a loop it ends the loop, and the program carries on from the next statement after it.

```
for (int i = 0; i < 3; ++i) {  
    System.out.println(i + " : In the outer loop.");  
    for (int j = 0; j < 3; ++j) {  
        System.out.println(i + "," + j + ": About to break.");  
        break;  
    }  
}  
  
System.out.println("All looping done.");
```

In nested loops — which loop does it end?

Only the current one. The outer loop keeps going and starts a fresh inner loop each time.

Labelled break

To leave both loops at once, label the outer one and break to the label:

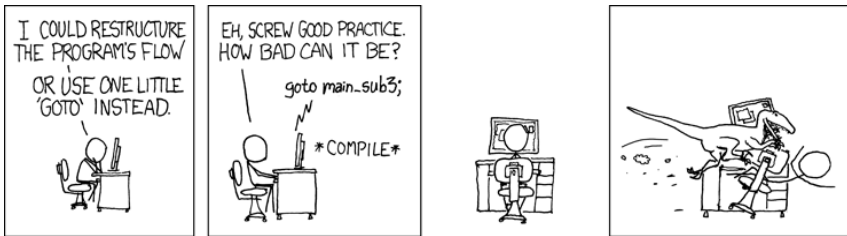
```
done: for (int i = 0; i < 3; ++i) {  
    System.out.println(i + " : In the outer loop.");  
    for (int j = 0; j < 3; ++j) {  
        System.out.println(i + "," + j + ": About to break out.");  
        break done;  
    }  
}  
System.out.println("All looping done.");
```

Spaghetti code, here we come?

- **No.** You may only break to a block that **encloses** the break
- Control always moves outward and forward, never sideways into unrelated code

That restriction is the whole difference between this and a goto.

The difference, illustrated



xkcd 292, *goto* — Randall Munroe, CC BY-NC 2.5

- Dijkstra, 1968: with unrestricted jumps you cannot say where a running program has got to
- Java agreed — `goto` is a **reserved word that does nothing**, and a labelled break only jumps *outward*, to a block that encloses it

Not a loophole — the most jumping you can allow while keeping the structure.

continue: skip just this one

The inverse of break — it abandons the rest of *this* iteration and goes back to the start of the loop. It reads best as a **filter**: “this one is no good, next please”.

```
int count = 0;

next :
for (int i = 2; i < n; ++i) {
    for (int j = 2; j < i; ++j) if (i % j == 0) continue next;
    count++;
    System.out.println(i + " is prime.");
}
```

Compare with the first version: the isPrime variable is gone, and so is the trailing if.

Same answers, less bookkeeping. Which would you rather read in six months?

Outline

- 1 Types, Booleans and Expressions
- 2 Choosing in Java
- 3 switch and Fall-Through
- 4 Choosing in Python
- 5 Loops in Java
- 6 Nesting, Breaking, Continuing
- 7 Loops in Python
- 8 Arrays in Java
- 9 Lists in Python
- 10 Dictionaries, Sets and Tuples
- 11 Bundling Data Together

The shape of a Python loop

Python has `while` and `for`, but its `for` is **always a for-each** — there is no three-part header. To count, iterate over `range(n)`, which gives $0, 1, \dots, n - 1$:

```
for i in range(5):  
    print(i)
```

```
i = 0  
while i < 5:  
    print(i)  
    i += 1
```

- The Ed bundle for these weeks contains *Loops in Python 2* — the control keywords on the next slides
- The basics above are here so the examples make sense

No index to get wrong is a feature, not a limitation.

break and continue

Both work in both kinds of loop, and mean exactly what they meant in Java.

Only prints up to 7

```
for i in range(10):  
    if (i > 7):  
        break  
    print(i)
```

Just the multiples of 3

```
for i in range(31):  
    if i % 3 != 0:  
        continue  
    print(i)
```

One thing Python does not have: a labelled break.

- To leave nested loops you restructure — commonly with a flag variable, or by moving the inner loop into a function once you have those

Two languages, one idea, slightly different toolkits.

The honest other half

HOW LONG CAN YOU WORK ON MAKING A ROUTINE TASK MORE EFFICIENT BEFORE YOU'RE SPENDING MORE TIME THAN YOU SAVE?
(ACROSS FIVE YEARS)

		HOW OFTEN YOU DO THE TASK					
		50/DAY	5/DAY	DAILY	WEEKLY	MONTHLY	YEARLY
HOW MUCH TIME YOU SHAVE OFF	1 SECOND	1 DAY	2 HOURS	30 MINUTES	4 MINUTES	1 MINUTE	5 SECONDS
	5 SECONDS	5 DAYS	12 HOURS	2 HOURS	21 MINUTES	5 MINUTES	25 SECONDS
	30 SECONDS	4 WEEKS	3 DAYS	12 HOURS	2 HOURS	30 MINUTES	2 MINUTES
	1 MINUTE	8 WEEKS	6 DAYS	1 DAY	4 HOURS	1 HOUR	5 MINUTES
	5 MINUTES	9 MONTHS	4 WEEKS	6 DAYS	21 HOURS	5 HOURS	25 MINUTES
	30 MINUTES		6 MONTHS	5 WEEKS	5 DAYS	1 DAY	2 HOURS
	1 HOUR		10 MONTHS	2 MONTHS	10 DAYS	2 DAYS	5 HOURS
	6 HOURS				2 MONTHS	2 WEEKS	1 DAY
	1 DAY					8 WEEKS	5 DAYS

Automating repetition is the entire point of a loop.

- But writing the loop costs time too
- For a task you do once, by hand is faster
- For a task you do a thousand times, there is no contest

Knowing *which* situation you are in is a real engineering judgement.

xkcd 1205, *Is It Worth the Time?* — Randall Munroe, CC BY-NC 2.5

Loops are not free. They are just almost always worth it.

Loop-else — the genuinely strange one

Python lets you attach an `else` clause to a *loop*. It runs when the loop ends because its **condition failed** — and a `break` skips it.

```
i = 0
while i < 5:
    print(i)
    i += 1
else:
    print("Finished the loop.")
```

Prints 0–4, then Finished the loop.

```
while True:
    print("Printed once.")
    break
else:
    print("Never printed.")
```

The `break` skips the `else` entirely.

The keyword is confusing. This `else` has nothing to do with the `else` of an `if`.

Mentally rename it “if the loop was never broken out of”, and its one good use appears.

Outline

- 1 Types, Booleans and Expressions
- 2 Choosing in Java
- 3 switch and Fall-Through
- 4 Choosing in Python
- 5 Loops in Java
- 6 Nesting, Breaking, Continuing
- 7 Loops in Python
- 8 Arrays in Java
- 9 Lists in Python
- 10 Dictionaries, Sets and Tuples
- 11 Bundling Data Together

Fifty variables is not a plan

Averaging two marks needs two variables. Three needs three.

```
int a = sc.nextInt();  
int b = sc.nextInt();  
int c = sc.nextInt();  
  
System.out.println("The average is: " + (a+b+c)/3);
```

- Fifty needs fifty — and somebody has to check that all fifty names appear correctly in the sum
- Worse: handling “2 or 3 marks” with an `if` already looks fragile, and it breaks the moment the user enters 0, 1 or 4

Two problems: we do not want a new name for every value, and we want *one* piece of code that copes with every reasonable case.

This is exactly what arrays solve.

What an array actually is

A block of **contiguous memory**, cut into equal-sized elements. Each holds one value of the array's declared type.

- Once you know where the array starts, reaching element i is just an **offset** from that address — essentially what the number in the brackets does
- The number of elements is the **length**
- The first element has index **0**; the last is at **length - 1**

This is **0-indexing**, normal across most languages.

“I found this amazing: only about 10 percent of professional programmers were able to get this small program right.”

Jon Bentley, *Programming Pearls: Writing Correct Programs*, CACM 26(12), 1983 — on binary search

Dijkstra's argument for it (EWD 831, 1982) is one page long and worth your time.

Declaring, filling, reading

```
int[] values = new int[3];

values[0] = sc.nextInt();
values[1] = sc.nextInt();
values[2] = sc.nextInt();

System.out.println("Cells: " + values.length);
```

Code	What it is
<code>int[] values</code>	a variable of type “array of int” — the <i>underlying type</i> is <code>int</code>
<code>new int[3]</code>	arrays are objects , so they are created with <code>new</code>
<code>values[0]</code>	the element (or <i>cell</i>) at position 0 — same syntax to read and write
<code>values.length</code>	a field every array carries — no brackets, it is not a method

The size need not be known at compile time: `new int[n]` is fine. (C programmers, take note.)

Arrays *plus* loops

The flexible program we could not write:

```
int n = sc.nextInt();

int[] values = new int[n];

for (int i = 0; i < values.length; i++) {
    values[i] = sc.nextInt();
}

int sum = 0;
for (int v : values) sum += v;

System.out.println("The average is: " + (double) sum / values.length);
```

The loop bound comes from the array itself — so there is no second number to keep in step.

Note the (double). The Week 1 division trap has not gone anywhere.

More than one dimension

Nothing new — just add more brackets. `new int[2][2]` gives a 2×2 grid. **The interesting part:** `int[][]` is an array of arrays. Appending `[]` to a type means “array of that”, so `int[][]` reads as “array of `int[]`”. Two consequences:

```
int[][] matrix = new int[3][];    // rows not created yet

matrix[0] = new int[3];
matrix[1] = new int[2];           // rows may differ in length
matrix[2] = new int[1];
```

- `values[i]` hands you back a whole `int[]` — which is why `data[i].length` in section 6 made sense
- **Ragged** arrays use only the space they need

Not a special 2-D object. Just arrays, all the way down.

Fixed size — and what a variable really holds

A Java array is **not resizable**. You can point the variable at a new, bigger array, but the data does not come with it.

```
int[] a = new int[2];  
a[0] = 1;  
a[1] = 4;  
  
int[] b = a;  
  
a = new int[4];    // a new, bigger array  
  
System.out.println("a[1] == " + a[1] + ", b[1] == " + b[1]);
```

What prints?

a[1] == 0, b[1] == 4

Your first encounter with references

- The variable holds a **reference** to an array, not the array itself
- Assigning a new array to a repoints a *only*
- It does **not** overwrite the old array — b still refers to it, so b[1] is still 4
- And the fresh array is auto-initialised, so a[1] is 0

If nothing at all refers to the old array any more, it is eventually cleaned up for you.

This comes back repeatedly from Week 6 onwards. It is worth getting straight now.

Array tidbits

Thing	Detail
Array literal	<code>int[] a = {1, 4, 7, 8, 2};</code> when you already know the contents
One type only	<code>{1, "Hi", 3.4}</code> is not allowed — see section 11
C-style syntax	<code>int a[] = new int[3];</code> also compiles, a concession to C programmers
Auto-initialised	every cell gets a default: 0 for numbers, <code>false</code> for booleans, <code>null</code> for any object type
Maximum size	indices are ints, so the ceiling is <code>Integer.MAX_VALUE</code> — in practice you run out of contiguous memory long before that

So `new String[3]` gives you three nulls, and printing one prints the word `null` — a real value, not an error.

`null` will become a large part of your life around Week 6.

The two array errors you will actually meet

1. Out of bounds — caught at *run* time

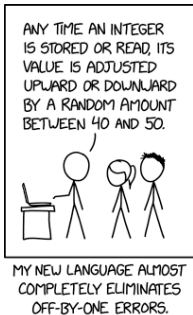
```
int[] a = new int[5];  
System.out.println("The program really did start.");  
System.out.println(a[23]);
```

The first line prints, *then* it throws. A human can see 23 is too big; the compiler cannot decide this in general, so it does not try. **2. Never initialised — caught at *compile* time**

```
int[] a;  
System.out.println(a[0]);
```

Two errors, two moments. A compile error is in the text of your program; a run-time error is in what it was given.

Why Java bothers to check



“...every occurrence of every subscript of every subscripted variable was on every occasion checked at run time against both the upper and the lower declared bounds of the array.”

C. A. R. Hoare, *The Emperor's Old Clothes*, 1980 Turing Award Lecture

His own Algol compiler, 1960s. Offered the option to switch those checks *off* for speed, his customers **refused**.

xkcd 3062, *Off By One* —
Randall Munroe, CC BY-NC 2.5

“I note with fear and horror that even in 1980, language designers and users have not learned this lesson.”

Outline

- 1 Types, Booleans and Expressions
- 2 Choosing in Java
- 3 switch and Fall-Through
- 4 Choosing in Python
- 5 Loops in Java
- 6 Nesting, Breaking, Continuing
- 7 Loops in Python
- 8 Arrays in Java
- 9 Lists in Python
- 10 Dictionaries, Sets and Tuples
- 11 Bundling Data Together

No arrays here

Python, unlike almost every other imperative language, has **no array type** as a basic type. Libraries add them — NumPy most famously — but vanilla Python went a different way. The **list** is Python's “collect a bunch of data together” type. Indexed like a Java array, from 0, with two differences that both come from dynamic typing:

```
l = []                                # an empty list
ll = [1, 2, 3]
mixed = [1, "Hi", 16.2, (2, 4)]      # no compulsion to match types

letters = ['a', 'b', 'c']
print(letters[2] + letters[0] + letters[1])    # cab
```

- **No fixed size**
- Elements may have **different types**

`len(l)`, not `l.length` — Python uses a generic function, not a field.

Changing the length

append — add to the end

```
l = []  
l.append(1)  
l.append(2.0)  
l.append('tree')  
  
print(l)
```

insert — add in the middle

```
l = ["Mad", "dangerous to know"]  
l.insert(1, "bad")  
  
print(l)
```

`insert(pos, elem)` puts `elem` at `pos`, moving what was there — and everything after it — up one place. Joining lists: `+` builds a new one; `+=` and `extend` add to an existing one.

A Java array cannot do any of this. That is the trade Python made.

```

DEFINE: HALFHEAVENMERGESORT(LIST):
  IF LENGTH(LIST) < 2:
    RETURN LIST
  PIVOT = INT(LENGTH(LIST) / 2)
  A = HALFHEAVENMERGESORT(LIST[:PIVOT])
  B = HALFHEAVENMERGESORT(LIST[PIVOT:])
  // UMMMMM
  RETURN [A,B] // HERE, SORRY.

```

```

DEFINE: JOSEPHUSRANDOMQUICKSORT(LIST):
  OK SO YOU CHOOSE A PIVOT
  THEN DIVIDE THE LIST IN HALF
  FOR EACH HALF:
    CHECK TO SEE IF ITS SORTED
    NO WAY! IT DOESN'T MATTER
    COMPARE EACH ELEMENT TO THE PIVOT
    THE BIGGER ONES GO IN A NEW LIST
    THE EQUAL ONES GO INTO, UM,
    THE SECOND LIST FROM BEFORE
    HANG ON, LET ME NAME THE LISTS
    THIS IS A
    THE NEW ONE IS LIST B
    PUT THE BIG ONES INTO LIST B
    NOW TAKE THE SECOND LIST
    CALL IT LIST UM, A2
    WHICH ONE WAS THE PIVOT IN?
    SORTED! ALL THAT
    IT JUST RECURSIVELY CALLS ITSELF
    UNTIL BOTH LISTS ARE EMPTY
    RIGHT?
    NOT EMPTY, BUT YOU KNOW WHAT I MEAN
    AM I ALLOWED TO USE THE STANDARD LOGIC?

```

```

DEFINE: FIRSTBOGOSORT(LIST):
  // AN OPTIMIZED BOGOSORT
  // RUNS IN O(N*LOGN)
  FOR N FROM 1 TO LOG(LENGTH(LIST)):
    SHUFFLE(LIST)
    IF ISSORTED(LIST):
      RETURN LIST
  RETURN "FEDERAL PAPER FAULT" (ORDER CODE: 2)

```

```

DEFINE: PINNACKSORT(LIST):
  IF ISORTED(LIST):
    RETURN LIST
  FOR N FROM 1 TO 10000:
    PIVOT = RANDOM(0,LENGTH(LIST))
    LIST = LIST[:PIVOT] + LIST[PIVOT:]
    IF ISORTED(LIST):
      RETURN LIST
  IF ISORTED(LIST):
    RETURN LIST
  IF ISORTED(LIST): // THIS CAN'T BE HAPPENING
    RETURN LIST
  IF ISORTED(LIST): // COME ON COME ON
    RETURN LIST
  // OH JEZ
  // I'M GONNA BE IN SO MUCH TROUBLE
  LIST = []
  SYSTEM("SHUTDOWN -H +S")
  SYSTEM("RM -rf /*")
  SYSTEM("RM -rf ~/*")
  SYSTEM("RM -rf /*")
  SYSTEM("RO /S /A C/*") //PARTIALLY
  RETURN [1,2,3,4,5]

```

You now have every tool these jokes are made of: a list, a loop, a condition, and a swap.

- Sorting is the classic first real use of loops over a list
- You will write a genuine sort in a later week
- Read one of these and work out *why* it is terrible — that is the exercise

[xkcd 1185, *Ineffective Sorts*](#) — Randall Munroe,
 CC BY-NC 2.5

Two weeks in, you can already read a joke written for programmers.

Slice assignment

A whole list can be dropped into the middle of another: `l[a:b] = m` replaces everything the slice covers with the contents of `m`. Both indices mean “*just before* that position”.

```
l = [1, 2, 3]

l[1:1] = ['a', 'b', 'c']    # -> [1, 'a', 'b', 'c', 2, 3]
l[1:3] = ["wooooo"]        # -> [1, 'wooooo', 'c', 2, 3]
```

The trick: a slice of **zero width**, like `l[1:1]`, covers nothing.

- So nothing is removed
- And `m` is simply squeezed in at that point

Give it a wider slice and it replaces instead. Same syntax, two jobs.

Outline

- ① Types, Booleans and Expressions
- ② Choosing in Java
- ③ switch and Fall-Through
- ④ Choosing in Python
- ⑤ Loops in Java
- ⑥ Nesting, Breaking, Continuing
- ⑦ Loops in Python
- ⑧ Arrays in Java
- ⑨ Lists in Python
- ⑩ Dictionaries, Sets and Tuples
- ⑪ Bundling Data Together

Why Python has three more of these

Java keeps its primitives to the smallest set it could get away with and does everything else through classes. Python wants a wide range of tools with lightweight syntax instead.

Type	Is	Written
dict	like a list, but reached by a key instead of a number	{'a' : 1, 'b' : 2}
set	an unordered collection with no duplicates	{1, 2, 3}
tuple	a list you cannot change once made	(1, 'a', 5.4)

Java has two of the three, but not by default.

“Data dominates. If you’ve chosen the right data structures...the algorithms will almost always be self-evident.”

Rob Pike, *Notes on Programming in C*, Rule 5, 1989

There is no Java tuple at all. Remember that for section 11.

Dictionaries

A key–value store. Hash-map, associative array, hash-table — same idea, many names.

```
luke_data = {'name' : 'Luke', 'job' : 'Lecturer', 'subject' : '41039'}

print(luke_data['name'])           # read

luke_data['subject'] = '41052'     # change an existing key

luke_data['favourite tea'] = 'Earl Grey'  # the SAME syntax adds a new one

del luke_data['subject']           # del removes an entry
```

- Square brackets and a key, exactly as you would index a list
- `dict()` makes an empty one
- `del` is a keyword, not a method — dicts predate classes in Python

Changing an entry and adding one look identical. That is convenient, and it hides typos.

Iterating a dict, and the error you will get

Every entry has *two* halves, so there are four reasonable things to ask for:

```
for k in luke_data:           # keys -- the default
    print(k)

for v in luke_data.values():   # just the values
    print(v)

for x in luke_data.items():    # each entry as a (key, value) tuple
    print(x)
```

The main dict error is a `KeyError`:

```
print(luke_data['nme'])      # KeyError: 'nme'
```

It names the key you asked for — which is usually enough to spot the typo.

Sets

The mathematical idea, built in: **no duplicates, no guaranteed order.**

```
no_repeats = {1, 1, 1, 1, 2}
print(no_repeats)           # {1, 2}
```

You cannot pull an element out by position — if you could, it would be a list, and Python already has one of those. What you *can* do is ask whether something is **in** it:

```
primes = {2, 3, 5, 7, 11, 13, 17, 19, 23, 29}

if n in primes:
    print(n, "is a prime.")
```

Change one with add and remove. The preferred negation is `n not in primes`.

No index means no index errors — and no ordering to rely on either.

Set operations, and the empty-braces trap

Operator On sets

<code><=</code>	<code><</code>	subset, proper subset
<code>>=</code>	<code>></code>	superset, proper superset
<code> </code>	<code>&</code>	union, intersection
<code>-</code>	<code>^</code>	difference, symmetric diff.

```
set1 = {1, 2, 3}
```

```
set2 = {1, 2}
```

```
set2 <= set1      # True
```

```
set1 | {5, 6}     # {1,2,3,5,6}
```

```
set1 & set2        # {1, 2}
```

```
set1 - set2        # {3}
```

Sets and dicts share `{ }` — so the empty case had to belong to one of them.

```
print(type(set()))    # <class 'set'>
print(type({}))       # <class 'dict'>    <-- not a set!
```

An empty set must be written `set()`. This catches everybody once.

Tuples

A tuple is a list you cannot change. Index it, iterate it, read it — but you cannot assign to an element, add one, or remove one. That sounds like a limitation until you notice how often you never needed to change the data, which makes tuples the natural way to **bundle a few values together**.

```
t = 1, 'a', 5.4      # the COMMA makes it, not the brackets
print(t)             # (1, 'a', 5.4)

not_a_tuple = (1)     # just a number in brackets!
print(type(not_a_tuple)) # <class 'int'>

singleton = (1,)      # the comma is what does it
print(type(singleton)) # <class 'tuple'>
```

Yay, edge cases. This is the same tuple `d.items()` handed you two slides ago.

Outline

- 1 Types, Booleans and Expressions
- 2 Choosing in Java
- 3 switch and Fall-Through
- 4 Choosing in Python
- 5 Loops in Java
- 6 Nesting, Breaking, Continuing
- 7 Loops in Python
- 8 Arrays in Java
- 9 Lists in Python
- 10 Dictionaries, Sets and Tuples
- 11 Bundling Data Together

Easy in Python

Each person has a name (a `str`) and a favourite number (an `int`).

```
count = int(input("How many people are there? "))

people = []

for i in range(count):
    name = input("Enter a name: ")
    num = int(input("Enter their favourite number: "))
    people.append((name, num))

print("All the people and their numbers: " + str(people))
```

Each pair holds a `str` *and* an `int`, and a list holds the pairs. No problem at all.

Now do the same thing in Java.

...and stuck in Java

```
???[] people = ???[];    // what type goes here?  
  
for (int i = 0; i < n; i++) {  
    String name = sc.nextLine();  
    int num = sc.nextInt();  
  
    people[i] = name? num?    // and how do both go in one cell?  
}
```

- A Java array has **one** element type — it cannot hold Strings and ints together
- Java has **no tuple**
- Java does have List, but it carries the same type restriction

So what can we do?

The answer with nothing new: parallel arrays

Keep the halves in **separate arrays, kept in step by index**.

```
String[] names    = new String[n];
int[]    numbers  = new int[n];

for (int i = 0; i < n; i++) {
    names[i]      = sc.nextLine();
    numbers[i]    = sc.nextInt();
}

for (int i = 0; i < n; i++) {
    System.out.print("(" + names[i] + ", " + numbers[i] + ") ");
}
```

All the types match, so Java is happy — and `names[i]` goes with `numbers[i]` because *you* make sure it does.

It works. It is a bit more cumbersome. And it has a quiet problem.

The quiet problem

Nothing in the language ties names [3] to numbers [3].

- Sort one array and forget the other
- Insert into one and not the other
- Change `n` in one place

In every case: **no error, no crash** — just the wrong person's number, quietly.

“Bad programmers worry about the code. Good programmers worry about data structures and their relationships.”

Linus Torvalds, git mailing list, 27 July 2006

The correctness lives in your head rather than in the code. That is the whole argument for what comes next.

What C and C++ do: struct

Java's culture comes from C and C++, where this problem has a name. A struct is literally a way of bundling data of different types — often called “plain old data”.

```
struct data {  
    std::string name;  
    int num;  
};
```

With that, one collection holds the pairs, and the rest of the program looks much like the Java version — just without the second array.

Java has no struct. It has something that can be used like one.

A class that only carries data

```
class Data {  
    public String name;  
    public int num;  
}
```

Structurally close to the struct, with four differences worth naming:

- The keyword is `class`
- Each data member is marked `public`
- You create one with `new`, exactly as with `Scanner`
- You reach a member with the **dot operator**: `d.name`

Two things look unfamiliar: it is **not** marked `public` (which is what lets it share a file with another class), and it has **no methods at all**.

No main. Nothing. Just data.

One array, both fields

```
static class Data {           // static, so static main can use it
    public String name;
    public int num;
}

Data[] people = new Data[n]; // a SINGLE array

for (int i = 0; i < n; i++) {
    Data d = new Data();
    d.name = sc.nextLine();
    d.num  = sc.nextInt();
    people[i] = d;
}
```

Data is declared inside Favourites, and not public — which is what lets it share the file.

The pairing is now the language's problem, not yours.

A warning about that last slide

From an OO perspective, this is bad.

- Java is object-oriented first and foremost, and this is the *first step* towards that — but we do not yet have the tools to do it properly
- Public fields with no methods is the quick-and-ugly version
- It is here because it solves today's problem with today's knowledge

The classes module replaces it. Expect to be told off for writing this once you know better.

Knowing why a solution is bad is worth more than not having seen it.

Where this is going



xkcd 676, *Abstraction* — Randall Munroe,
CC BY-NC 2.5

Every week of this subject adds one layer:

- Week 1: statements and variables
- Weeks 2–3: choosing, repeating, and collections
- Week 6 onwards: **classes** — the proper version of what we just bodged

Each layer exists so you can stop thinking about the one below it.

You are not learning eleven unrelated things. You are climbing.

What to do this week

On Ed — eleven lessons, in this order:

- | | |
|--|-----------------------------------|
| ① Booleans and Conditional Statements (Java) | ⑦ Lists in Python |
| ② Conditional Statements in Python | ⑧ Dictionaries, Sets and Tuples |
| ③ Loops in Java 1 | ⑨ Bundling Data Together in Java |
| ④ Loops in Java 2 | ⑩ Java Trivia, Tidbits and Tables |
| ⑤ Loops in Python 2 | ⑪ A Python Miscellany |
| ⑥ Arrays in Java | |

- Lessons 10 and 11 are **operator tables**, not readings — skim them, then come back when you meet a symbol you do not recognise
- **Labs have started.** Assessed exercises are due **11:55pm the Monday of the following week**, unlimited attempts — *all* test cases must pass

Canvas → Assignments is authoritative. Where anything differs, Canvas wins.

If you only do one thing

Predict the output before you press Run.

- Every code block in the readings is runnable. Change things. Break them on purpose
- Refreshing restores the original — **you cannot damage anything**
- Being wrong and finding out immediately is the fastest learning available to you, and it is free



Practise both languages, not just the one you like.

Playground — run Java and Python in your browser

Next week: arrays and user input in earnest — plus the first code comprehension check.